

COMP0264 Generative AI

Exam Revision Notes: Transformers

Auto-generated from Lecture Slides & Recordings

UCL 2025–26

This document covers lectures from 19 Jan – 03 Feb 2026 (5 sessions).

Sources: GenAI2526_Transformers.pdf, TransformerTutorialNotes (Parts 1 & 2), and lecture recordings.

Contents

1 Overview	2
2 Tokens and Input Representation	2
3 Embedding Spaces and Representation Learning	3
4 Self-Attention: From Hard Attention to Learned Attention	3
4.1 Attention Coefficients	3
4.2 Hard (Naïve) Self-Attention	4
4.3 Why Hard Attention Is Insufficient	4
5 Query, Key, Value: Learnable Self-Attention	5
6 Scaled Self-Attention	6
7 Multi-Head Attention	7
7.1 Head Redundancy and Collapse	7
8 The Complete Transformer Layer	8
8.1 Residual Connections	8
8.2 Layer Normalisation	8
8.3 Feed-Forward Network (MLP)	9
8.4 Complete Transformer Layer	9
8.5 Pre-Norm Variant	10
9 Positional Encoding	10
9.1 Requirements for Positional Encoding	10
9.2 Sinusoidal Positional Encoding	11
9.3 Learned Positional Encoding	11
9.4 RoPE: Rotary Position Embedding	11
10 Summary: Transformer Architecture Hierarchy	12
11 Training: Key Concepts	12
12 Self-Attention Properties and Linearity	12

13 Potential Exam Questions	13
14 References	14

Shuhao Wang

1 Overview

Transformers represent one of the most important developments in deep learning, introduced in the 2017 paper “Attention Is All You Need” (Vaswani et al.). They are based on the *attention* mechanism, which allows a network to assign different weights to different inputs, capturing inductive biases related to sequential data. The transformer architecture forms the basis of all modern large language models (GPT, Claude, LLaMA, etc.) and multimodal models.

LECTURE INSIGHT:

Why Transformers Before GANs/VAEs/Diffusion The lecturer chose to cover Transformers early in the module because: (1) Transformers are somewhat independent of GANs/VAEs/Diffusion, (2) the coursework project would be about transformers and foundational models, so students need time to study this material, and (3) this is arguably the major innovation in AI in the past decade. (Mentioned in Lectures 01-19, 02-02)

LECTURE INSIGHT:

The “Attention Is All You Need” Paper The lecturer described this paper as “one of the most difficult papers to understand” and advised students **not** to start their study from the original paper. He recommended instead using explanatory papers and blog posts, and said he would provide pointers. The paper was published openly by Google, which allowed competitors to build on it—an interesting strategic choice. (Mentioned in Lecture 01-19)

2 Tokens and Input Representation

Definition: Tokens and Input Matrix

The input data to a transformer is a set of vectors $\{\mathbf{x}_n\}$ of dimensionality D , where $n = 1, \dots, N$. These vectors are called **tokens**.

Examples of tokens: words in a sentence, patches in an image, amino acids in a protein, etc.

We combine these into a matrix $\mathbf{X}$ of dimensions $N \times D$, where the n -th row corresponds to the token vector $\mathbf{x}_n^T$.

LECTURE INSIGHT:

What Is a Token? Each token is not a single number—it is a **vector** of dimension D . Think of each token as a point in a D -dimensional embedding space. A single instance of input (e.g., one sentence, one image, one protein) is represented by the matrix $\mathbf{X}$; during training, we process many such instances. (Mentioned in Lectures 01-19, 01-26)

Definition: Transformer Layer (High Level)

The fundamental block of a transformer is a function that takes a data matrix $\mathbf{X}$ as input and creates a transformed matrix:

$$\tilde{\mathbf{X}} = \text{TransformerLayer}[\mathbf{X}]$$

We stack multiple transformer layers in succession to construct deep networks capable of learning rich representations.

LECTURE INSIGHT:

What Does a Transformer Actually Do? A transformer is “a very complicated function that transforms a vector into another vector that is useful for something.” It produces a good representation for a downstream task. The lecturer stressed: there is nothing magical about transformers—they are a well-engineered matrix transformation that is good for training (due to invariance properties) and scales well to large token sequences. It is **not** necessarily the final answer to AGI. (Mentioned in Lecture 01-19)

3 Embedding Spaces and Representation Learning

Tutorial Note: What Transformers Transform

A transformer does not primarily transform a single vector. It transforms a **set (or sequence) of vectors jointly**. It maps a set of input tokens $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ in an embedding space to another set $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$ in an embedding space with **richer semantics**.

Each layer of a neural network maps inputs from one vector space to a new one. Across layers, the network learns to re-encode information, untangle factors of variation, and make useful features separable.

LECTURE INSIGHT:

Geometric View of Transformers Because of nonlinearities, a neural network performs a **nonlinear warping** of representation spaces—stretching, folding, compressing the space so that the desired output structure emerges. Deep learning is fundamentally **representation learning**: learning a hierarchy of representations where high-level concepts are built from lower-level ones. Training reshapes the representation geometry so that sequences with similar continuation distributions become nearby, tokens that play similar roles cluster together, and attention learns stable geometric patterns (subspaces form, relational directions emerge). (Mentioned in Tutorial 01-27)

STUDENT Q&A:

Is the transformer mapping within the same space or to a different space? The lecturer clarified: we are moving points around within the embedding space. The transformer gives a representation that is more meaningful—words that appear in similar contexts end up close to each other. Think of it as moving points in space so that contextually related tokens are geometrically close. The output space has the same dimensionality D as the input (which enables stacking layers), but the positions of points have changed to reflect contextual meaning. (Mentioned in Lecture 01-19)

4 Self-Attention: From Hard Attention to Learned Attention

4.1 Attention Coefficients

Definition: Attention as Weighted Combination

Suppose we have input tokens $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ and want to map them to output tokens $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$. The value of $\mathbf{y}_n$ should depend not just on $\mathbf{x}_n$ but on **all** vectors in the set.

We define $\mathbf{y}_n$ as a linear combination:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

where a_{nm} are **attention coefficients** subject to constraints:

$$a_{nm} \geq 0 \quad \text{and} \quad \sum_{m=1}^N a_{nm} = 1$$

LECTURE INSIGHT:

Geometric Intuition for Attention Since $\mathbf{y}_n$ is a **convex combination** of vectors, each output geometrically moves towards the vectors with higher attention coefficients. If you give more weight to one vector, the result is pulled closer to that vector in embedding space.

Example: if “bank” appears with “loan,” and we give high attention to “loan,” then the output vector for “bank” moves close to “loan” in the embedding space, thereby encoding the financial meaning. If instead “bank” appears with “river,” the output moves toward “river.” This is how context disambiguates meaning. (Mentioned in Lectures 01-19, 01-26)

4.2 Hard (Naïve) Self-Attention

Hard Self-Attention

A simple approach uses a similarity matrix $\mathbf{S} = \mathbf{X}\mathbf{X}^T$, where each element is the dot product $s_{nm} = \mathbf{x}_n^T \mathbf{x}_m$. To enforce the constraints, we apply softmax:

$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

In matrix form:

$$\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T] \mathbf{X}$$

where $\text{Softmax}(\mathbf{L})$ exponentiates every element of matrix $\mathbf{L}$ and normalises each row independently to sum to 1.

LECTURE INSIGHT:

Dot Product as Similarity The dot product $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$ measures similarity via the angle between vectors. This is the same idea behind **cosine similarity** used in information retrieval for 40–50 years. If two vectors are the same, the dot product is maximal; if orthogonal, it is zero. (Mentioned in Lecture 01-19)

4.3 Why Hard Attention Is Insufficient

Problems with Hard Attention

Two key problems:

1. The transformation from $\mathbf{x}_n$ to $\mathbf{y}_n$ is **fixed** and has no capacity to learn from data—it is just the input multiplied by itself.
2. Each feature value in $\mathbf{x}_n$ has the **same influence** on the attention coefficients.

We need: (a) **learnable** attention weights, and (b) **asymmetric** attention (since $\mathbf{X}\mathbf{U}\mathbf{U}^T\mathbf{X}^T$ is symmetric).

LECTURE INSIGHT:

Why Asymmetry Matters: The Cake Example Consider 5 tokens: **cake**, **sugar**, **milk**, **flour**, **eggs**. The attention from ingredients to recipe name should be strong (many-to-one), but from recipe name to individual ingredients should be weak and distributed. A symmetric attention matrix cannot model this asymmetry. (Mentioned in Lectures 01-26, slides)

5 Query, Key, Value: Learnable Self-Attention

Definition: Query, Key, Value Matrices

To achieve learnable, asymmetric attention, we introduce three separate weight matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}, \quad \mathbf{W}^{(q)} \in \mathbb{R}^{D \times D_k} \quad (1)$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}, \quad \mathbf{W}^{(k)} \in \mathbb{R}^{D \times D_k} \quad (2)$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}, \quad \mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v} \quad (3)$$

where $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, $\mathbf{W}^{(v)}$ are **learned during training**.

The generalised self-attention formula becomes:

$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}[\mathbf{Q}\mathbf{K}^T] \mathbf{V}$$

where $\mathbf{Q}\mathbf{K}^T$ has dimension $N \times N$ and $\mathbf{Y}$ has dimension $N \times D_v$.

Typical choice: $D_k = D$ and $D_v = D$ so that input and output have the same size, enabling residual connections and stacking of transformer layers.

LECTURE INSIGHT:

Why Q, K, V? The requirements that led to Q, K, V: (1) we need **learnable** weights (not fixed dot products), (2) we need **asymmetric** attention (not symmetric like $\mathbf{X}\mathbf{U}\mathbf{U}^T\mathbf{X}^T$), and (3) we need the ability to give **different weights to different features**. Using separate matrices for Q, K, V satisfies all three requirements.

The lecturer stressed: “Don’t learn this by heart. Reason about it—the derivation is logical.” (Mentioned in Lectures 01-26, 02-02)

LECTURE INSIGHT:

Analogy with Information Retrieval The Q/K/V terminology comes from Information Retrieval: Query = what you search for, Key = index/keywords of each document, Value = the actual document content. However, the lecturer noted this analogy is “a bit confusing” for transformers and recommended thinking in terms of the mathematical derivation from asymmetric learnable attention instead. (Mentioned in Lecture 01-26)

STUDENT Q&A:

Can bias parameters be included in Q, K, V? Yes, you can have $\mathbf{Q} = \mathbf{XW}^{(q)} + \mathbf{b}^{(q)}$, etc. The bias can be absorbed into the weight matrices as a simplification. In the slides, bias is omitted for notational clarity. (Mentioned in Lecture 01-26)

STUDENT Q&A:

Why is Q similar to K in form? At first glance, Q and K look structurally identical (both are $\mathbf{X}$ times a weight matrix). The key difference is that they have **different learned parameters**, which breaks the symmetry. During training, Q learns “what to look for” and K learns “what to advertise,” resulting in asymmetric attention. (Mentioned in Lectures 01-26, 01-27)

6 Scaled Self-Attention

Definition: Scaled Dot-Product Attention

The gradients of the softmax function become exponentially small for large inputs (the softmax saturates). To avoid this, we rescale:

$$\mathbf{Y} = \text{Softmax} \left[\frac{\mathbf{QK}^T}{\sqrt{D_k}} \right] \mathbf{V}$$

Derivation of $\sqrt{D_k}$ Scaling

Assumptions: each element of $\mathbf{Q}$ and $\mathbf{K}$ is independent with zero mean and unit variance. Under these assumptions:

1. Each product $Q_i K_i$ has variance = 1.
2. The dot product is a sum of D_k such terms.
3. The variance of a sum of independent terms is the sum of variances = D_k .
4. The standard deviation is $\sqrt{D_k}$.

By dividing by $\sqrt{D_k}$, we normalise the dot products to have unit variance, keeping the softmax in a well-behaved regime where gradients don't vanish.

LECTURE INSIGHT:

Why Unit Variance Assumption? The assumption of zero mean and unit variance comes from how we typically initialise the weight matrices. The scaling ensures we stay in the region where the softmax function has meaningful gradients rather than saturating to 0 or 1. Without scaling, for large D_k , the dot products grow large and the softmax outputs become nearly one-hot, killing gradients. (Mentioned in Lecture 01-26)

EXAM ALERT:

Scaling Factor Derivation The lecturer explicitly asked students to derive why the variance of the dot product is D_k and said this type of reasoning question is representative of what appears on the final exam. Be prepared to show: $\text{Var}(\mathbf{q}^T \mathbf{k}) = \sum_{i=1}^{D_k} \text{Var}(q_i k_i) = D_k \cdot 1 = D_k$. (Mentioned in Lecture 01-26)

7 Multi-Head Attention

Definition: Multi-Head Attention

A single attention head captures one pattern. To capture multiple patterns simultaneously (e.g., tense, vocabulary, style, semantic role), we use H heads in parallel. Each head h has its own weight matrices:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(q)}, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(k)}, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(v)} \quad (4)$$

$$\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{Softmax}\left[\frac{\mathbf{Q}_h\mathbf{K}_h^T}{\sqrt{D_k}}\right] \mathbf{V}_h \quad (5)$$

The heads are concatenated and linearly projected:

$$\mathbf{Y}(\mathbf{X}) = \text{Concat}[\mathbf{H}_1, \mathbf{H}_2, \dots, \mathbf{H}_H] \mathbf{W}^{(o)}$$

where each $\mathbf{H}_h$ has dimension $N \times D_v$, the concatenated matrix has dimension $N \times HD_v$, and $\mathbf{W}^{(o)}$ has dimension $HD_v \times D$.

Key choice: $D_v = D/H$ so that $HD_v = D$ and the output has dimension $N \times D$ (same as input).

LECTURE INSIGHT:

Why $D_v = D/H$? This choice ensures: (1) after concatenation, the total dimension is $HD_v = D$, matching the input dimension; (2) the output matrix $\mathbf{W}^{(o)}$ maps $HD_v \rightarrow D$ (which becomes $D \rightarrow D$, i.e., a square matrix); (3) we can stack transformer layers since input and output dimensions match; (4) each head works in a compressed subspace of dimension D/H , which can be seen as a form of compression that encourages specialisation. (Mentioned in Tutorial 01-27)

LECTURE INSIGHT:

Why Concatenate Instead of Sum? Concatenation is preferred over summation because summing would cause heads to interfere with each other, collapsing specialised patterns. Concatenation delays “mixing” until the final linear transformation $\mathbf{W}^{(o)}$, preserving each head’s independent subspace. (Mentioned in Tutorial Notes First Part)

7.1 Head Redundancy and Collapse

Head Collapse Problem

At first glance, all heads attend over the same tokens with the same computational form. Is there redundancy?

In practice, each head learns **different patterns** because:

1. Different random weight initialisation breaks symmetry immediately.
2. Gradient competition: if two heads encode the same pattern, only one receives strong gradients.
3. Data complexity makes identical solutions unlikely.

Preventing Head Collapse

Techniques to guarantee no collapse:

1. Orthogonality penalties between heads
2. Diversity regularisation

3. Mutual information maximisation
4. Cosine-distance penalties

However, in practice, simple random initialisation and gradient competition are usually sufficient.

STUDENT Q&A:

What guarantees heads learn different things? The lecturer said: “Not really guaranteed.” But different initialisation + gradient competition provide “soft pressure.” If two heads become identical, one is redundant, receives weaker gradients, and drifts. Data complexity makes identical solutions unlikely. The $\mathbf{W}^{(o)}$ projection can also rebalance the contribution of each head. (Mentioned in Lectures 01-26, 01-27)

EXAM ALERT:

Multi-Head Attention: Key Exam Question The lecturer explicitly stated: “Why do you need multi-head attention?” is a type of exam question. The answer should be concise: “Different heads can capture different attention patterns from the same input (e.g., tense, semantics, style). This is more efficient than a single head.” Do **not** write two pages re-deriving the transformer—be precise and to the point. (Mentioned in Lecture 02-02)

8 The Complete Transformer Layer

8.1 Residual Connections

Residual Connection

When stacking many transformer layers, we encounter the vanishing gradient problem. The solution is to add a **residual connection**:

$$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}]$$

where $\mathbf{Y}(\mathbf{X}) = \text{Concat}[\mathbf{H}_1, \dots, \mathbf{H}_H] \mathbf{W}^{(o)}$ is the multi-head self-attention output.

8.2 Layer Normalisation

Layer Normalisation

Layer normalisation is preferred over batch normalisation in transformers because batch normalisation fails with: varying sequence lengths, small batches (instability), and computation across multiple GPUs.

For each sample n across all M hidden units:

$$\mu_n = \frac{1}{M} \sum_{i=1}^M a_{ni} \quad (6)$$

$$\sigma_n^2 = \frac{1}{M} \sum_{i=1}^M (a_{ni} - \mu_n)^2 \quad (7)$$

$$\hat{a}_{ni} = \frac{a_{ni} - \mu_n}{\sqrt{\sigma_n^2 + \delta}} \quad (8)$$

Batch normalisation normalises across the mini-batch (column-wise), while **layer normalisation** normalises across hidden units within each sample (row-wise).

8.3 Feed-Forward Network (MLP)

Non-Linearity via MLP

The self-attention outputs are constrained to lie in the subspace spanned by the input vectors (since they are linear combinations). To introduce **non-linearity**, we apply a multi-layer perceptron (MLP) with D inputs and D outputs, applied independently to each token vector (each row of $\mathbf{Z}$).

This preserves the ability to process sequences of variable length.

STUDENT Q&A:

Why do we need non-linearity? The lecturer said this is an important exam-style question. Without non-linearity, stacking many layers of linear transformations is equivalent to a single linear transformation. Non-linearity allows the network to capture more complex manifolds and decision boundaries—it makes the model more **expressive**. The only non-linearity in the attention mechanism itself is the softmax; the MLP adds crucial additional non-linearity. (Mentioned in Tutorial 01-27)

8.4 Complete Transformer Layer

Complete Transformer Layer Formula

The full transformer layer with two residual connections and layer normalisation:

$$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}] \quad (9)$$

$$\tilde{\mathbf{X}} = \text{LayerNorm}[\text{MLP}(\mathbf{Z}) + \mathbf{Z}] \quad (10)$$

where $\mathbf{Y}(\mathbf{X})$ is the multi-head self-attention output.

A **transformer architecture** is obtained by stacking several such transformer layers.

8.5 Pre-Norm Variant

Pre-Norm Transformer Layer

A common variation applies layer normalisation **before** the attention/MLP blocks instead of after:

$$\mathbf{X}' = \text{LayerNorm}[\mathbf{X}] \quad (11)$$

$$\mathbf{Z} = \mathbf{Y}(\mathbf{X}') + \mathbf{X} \quad (12)$$

$$\mathbf{Z}' = \text{LayerNorm}[\mathbf{Z}] \quad (13)$$

$$\tilde{\mathbf{X}} = \text{MLP}(\mathbf{Z}') + \mathbf{Z} \quad (14)$$

This has been shown experimentally to be more stable during training.

LECTURE INSIGHT:

Why Pre-Norm Works Better The lecturer speculated: by normalising before passing through the attention/MLP block, you prevent large variations from being magnified by subsequent operations. This reduces the probability of exploding/vanishing gradients. However, there is no formal proof—this is an empirical finding. (Mentioned in Lecture 02-02)

9 Positional Encoding

The Permutation Invariance Problem

The transformer has the property that permuting the order of input tokens (rows of $\mathbf{X}$) results in the same permutation of the output rows. This is because attention computes a weighted sum that is invariant to ordering.

This is a major limitation for sequential data. Example:

- “Nvidia is up today, instead Microsoft is down”
- “Nvidia is down today, instead Microsoft is up”

Same tokens, different order $\Rightarrow$ completely different semantics, but the transformer would produce the same attention weights.

9.1 Requirements for Positional Encoding

Ideal Positional Encoding Properties

1. **Bounded:** values should not grow with sequence length.
2. **Relative position:** should consistently express the distance between any two tokens, regardless of absolute position.
3. **No dimensionality increase:** should not enlarge the embedding space (concatenation is bad due to curse of dimensionality).

LECTURE INSIGHT:

Why Not Simple Approaches? The lecturer discussed and dismissed: (1) Integer indices (1, 2, 3, ...): magnitude grows and corrupts the embedding; doesn't generalise to longer sequences. (2) Normalised to $[0, 1]$: depends on sequence length, so the encoding is not unique. (3) Concatenation with the token: increases dimensionality, leading to curse of dimensionality. The solution is to **add** a position vector to the token embedding. (Mentioned in Lecture 02-02)

9.2 Sinusoidal Positional Encoding

Sinusoidal Positional Encoding (“Attention Is All You Need”)

For position n , the encoding vector $\mathbf{r}_n$ has components:

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right) & \text{if } i \text{ is odd} \end{cases}$$

where L is related to the maximum sequence length and D is the embedding dimension. The modified input is:

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

Key Property: Relative Position

For any fixed offset k , the encoding at position $n+k$ can be represented as a **linear combination** of the encoding at position n :

$$\mathbf{r}_{n+k} = f(k) \cdot \mathbf{r}_n \quad (\text{linear combination, coefficients depend only on } k)$$

This follows from the trigonometric identities $\sin(a+b)$ and $\cos(a+b)$. The coefficients do not depend on the absolute position n , only on the offset k —so the network can learn to attend to **relative** positions.

LECTURE INSIGHT:

Practical Advice on Positional Encoding The lecturer strongly recommended: take Python/Matlab, plot the sinusoidal encodings for different positions and dimensions, and visualise the heatmap. This builds intuition much faster than staring at formulas. The heatmap shows characteristic stripe patterns of varying frequencies across the embedding dimension. (Mentioned in Lectures 02-02, 02-03)

9.3 Learned Positional Encoding

Learned Positional Encoding

An alternative is to learn a vector of positional weights during training.

Pro: Since parameters are not shared during training, the tokens are not invariant under permutation.

Con: Does not generalise to sequences longer than those seen during training.

9.4 RoPE: Rotary Position Embedding

RoPE (Rotary Position Embedding)

Instead of adding positional embeddings to token embeddings, RoPE **rotates** the query and key vectors by an angle that depends on their position in the sequence. This captures relative positions through the properties of rotation matrices.

RoPE is used in LLaMA, Mistral, and Qwen models.

LECTURE INSIGHT:

RoPE: Not Required for Exam The lecturer said: “RoPE is a bit exotic. I’m not expecting you to study it for the exam.” However, it is good to know it exists and that it is used in modern models like LLaMA. The key idea is encoding relative positions through rotation rather than addition. (Mentioned in Lecture 02-03)

10 Summary: Transformer Architecture Hierarchy

Architecture Hierarchy

1. **Scaled Dot-Product Attention:** $\text{Softmax}\left[\frac{\mathbf{QK}^T}{\sqrt{D_k}}\right] \mathbf{V}$
2. **Attention Head:** one instance of scaled attention with learned $\mathbf{W}^{(q)}, \mathbf{W}^{(k)}, \mathbf{W}^{(v)}$
3. **Multi-Head Attention:** H heads concatenated + linear projection $\mathbf{W}^{(o)}$
4. **Transformer Layer:** Multi-Head Attention + Residual + LayerNorm + MLP + Residual + LayerNorm
5. **Transformer Architecture:** stack of Transformer Layers + Positional Encoding
6. **Foundation Models** (LLMs, VLMs): Transformer Architecture + training objective + data

11 Training: Key Concepts

LECTURE INSIGHT:

Training Objective The transformer architecture itself does not specify the training objective. For language modelling, the typical objective is **next-token prediction**: maximise the likelihood of the correct next token given all previous tokens. The loss function is **cross-entropy**. Training uses standard backpropagation with stochastic gradient descent.

The weights being learned include: all $\mathbf{W}_h^{(q)}, \mathbf{W}_h^{(k)}, \mathbf{W}_h^{(v)}$ for each head h , the projection matrix $\mathbf{W}^{(o)}$, and the MLP parameters. (Mentioned in Tutorial 01-27)

LECTURE INSIGHT:

Why So Many Parameters? If you want to capture one poem, you need few parameters. If you want all 19th-century British romantic poetry, you need more. If you want all of the internet, you need ~ 1 trillion. The number of parameters must match the **variation** in the training data. More parameters = more expressiveness = ability to capture more complex manifolds. This is the mathematical basis for the scaling laws observed in LLMs. (Mentioned in Lectures 01-19, 01-26)

12 Self-Attention Properties and Linearity

Linearity of Self-Attention

The self-attention mechanism is **linear** except for the softmax nonlinearity:

- $\mathbf{Q} = \mathbf{XW}^{(q)}$ — linear
- $\mathbf{K} = \mathbf{XW}^{(k)}$ — linear
- $\mathbf{V} = \mathbf{XW}^{(v)}$ — linear
- $\mathbf{QK}^T$ — bilinear
- $\text{Softmax}(\cdot)$ — **nonlinear**

- Multiplication by $\mathbf{V}$ — linear

The MLP block adds the crucial additional non-linearity needed for expressiveness.

EXAM ALERT:

Is Attention Linear or Nonlinear? This was explicitly flagged as an exam-type question. The answer: the attention mechanism is mostly linear—the only nonlinearity is the softmax. The MLP in the transformer layer provides additional nonlinearity. Without non-linearity, stacking layers would collapse to a single linear transformation. (Mentioned in Tutorial 01-27)

13 Potential Exam Questions

EXAM ALERT:

1. What are transformers and what do they transform? A transformer transforms a set/sequence of token vectors jointly, mapping them from one embedding space to another with richer, context-aware semantics. It does **not** transform a single vector in isolation. (Lecturer emphasis: Tutorial 01-27)

EXAM ALERT:

2. Why do we need learnable attention (Q, K, V)? Hard attention ($\mathbf{XX}^T$) is fixed, cannot learn from data, gives equal influence to all features, and produces symmetric attention. Q, K, V introduce learnable, asymmetric attention where different features can have different importance. (Lecturer emphasis: Lectures 01-26, 02-02)

EXAM ALERT:

3. Why the $\sqrt{D_k}$ scaling? Under zero-mean, unit-variance assumptions for Q and K elements, the dot product has variance D_k . Dividing by $\sqrt{D_k}$ normalises to unit variance, preventing softmax saturation and vanishing gradients. (Lecturer emphasis: Lecture 01-26)

EXAM ALERT:

4. Why multi-head attention? Different heads can capture different attention patterns simultaneously (e.g., semantics, tense, style). This is more expressive than a single head. The answer should be concise—3 lines maximum. (Lecturer emphasis: Lecture 02-02)

EXAM ALERT:

5. Why $D_v = D/H$? So that after concatenating H heads, the total dimension is $HD_v = D$, matching the input dimension. This enables: (a) stacking layers, (b) residual connections, (c) preserving embedding space interpretation. (Lecturer emphasis: Tutorial 01-27)

EXAM ALERT:

6. Why layer normalisation over batch normalisation? Batch normalisation fails with: variable sequence lengths, small batch sizes (instability), and multi-GPU computation. Layer normalisation operates per-sample across features, avoiding these issues. (From slides)

EXAM ALERT:

7. Why positional encoding? The attention mechanism is invariant to token permutation (it's a weighted sum). Positional encoding breaks this invariance by injecting position information. Sinusoidal encoding is bounded, captures relative positions (via trigonometric identities), and does not increase dimensionality. (Lecturer emphasis: Lecture 02-02)

EXAM ALERT:

8. Why is non-linearity needed in the transformer? Self-attention outputs are constrained to the subspace spanned by input vectors (linear combinations). The MLP introduces non-linearity, enabling the network to learn more complex manifolds and making stacking layers meaningful (otherwise multiple linear layers collapse to one). (Lecturer emphasis: Tutorial 01-27)

EXAM ALERT:

Exam Strategy Advice from Lecturer “The final exam will have questions like ‘Why do you need multi-head attention?’ Answer in a **concise, precise, to-the-point** way—3 lines is enough. If you write two pages discussing the whole transformer, it shows you didn't understand the question. You have 2 hours with many questions; conciseness is key.” (Mentioned in Lecture 02-02)

14 References

Listed in Slides

- Chapter 12 of C. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.
- A. Vaswani et al. “Attention Is All You Need.” *NeurIPS*, 2017.
- Layer Normalisation: Ba, Kiros, Hinton. arXiv:1607.06450.
- RoPE: Su et al. *Neurocomputing*, 2024.

Mentioned in Lectures (Not on Slides)

- The lecturer mentioned he would share explanatory papers that “interpret” the original Attention Is All You Need paper (described as interpretations of a “sacred text”).
- Word2Vec embedding (mentioned as classic pre-transformer embedding technique for comparison).
- The HuggingFace Transformers library (**transformers**) was introduced for practical coding (Lecture 02-03).
- Information bottleneck concept (relevant to understanding head dimension compression).